Terraform Challenge

Create and open the terraform-docker-cricbuzz folder.

Create main.tf and add the below code

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Security Group
resource "aws_security_group" "web_sg" {
  name        = "cricbuzz-web-sg"
  description = "Allow web traffic on port 8080"

  ingress {
    description = "HTTP application access"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Cricbuzz-Web-SG"
  }
}

# EC2 Instance
resource "aws_instance" "cricbuzz" {
  ami                    = "ami-0461b1436c9b3c1a3"
  instance_type          = "t2.micro"
  associate_public_ip_address = true

  vpc_security_group_ids = [
    aws_security_group.web_sg.id
  ]

  user_data = file("${path.module}/user_data.sh")

  tags = {
    Name = "Terraform-Cricbuzz-Server"
  }
}
```

Output public IP

output "public_ip" {

value = aws_instance.cricbuzz.public_ip

}

this code does:

- Configures AWS in N. Virginia.

- Creates a Security Group allowing port 8080.

- Creates a t2.micro Ubuntu EC2 instance.

- Passes a startup script to EC2.

- Displays the public IP after deployment.

We'll create user_data.sh next. That script will install Docker, Docker Compose and Nginx, deploy the application, configure reverse proxy logging and set up Logrotate.

Create `user_data.sh`

Inside your project folder:

1. **Right-click the folder.**
2. **Select New File.**
3. **Name it `user_data.sh`.**
4. **Paste the following complete script:**

#!/bin/bash

set -euxo pipefail

Update Ubuntu

apt-get update -y

Install required packages

apt-get install -y docker.io docker-compose-v2 nginx git logrotate

Enable Docker

```
systemctl enable docker
systemctl start docker

# Enable Nginx
systemctl enable nginx

# Clone Cricbuzz project
mkdir -p /opt/cricbuzz
git clone https://github.com/betawins/cricbuzz-docker-hello-world.git /opt/cricbuzz/app

cd /opt/cricbuzz/app

# Create Docker Compose file
cat > docker-compose.yml <<'EOF'
services:
  cricbuzz:
    build: .
    container_name: cricbuzz-app
    ports:
      - "127.0.0.1:8000:8000"
    restart: unless-stopped
EOF

# Build and start Docker application
docker compose up -d --build

# Configure Nginx reverse proxy
cat > /etc/nginx/sites-available/cricbuzz <<'EOF'
server {
```

```
    listen 8080;
    server_name _;

    access_log /var/log/cb.log combined;
    error_log /var/log/cb-error.log warn;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

# Enable Nginx configuration
rm -f /etc/nginx/sites-enabled/default
ln -s /etc/nginx/sites-available/cricbuzz /etc/nginx/sites-enabled/cricbuzz

nginx -t
systemctl restart nginx

# Configure Logrotate
cat > /etc/logrotate.d/cricbuzz <<'EOF'
/var/log/cb.log {
    daily
    rotate 7
    compress
```

missingok

notifempty

create 0640 www-data adm

sharedscripts

postrotate

nginx -s reopen

endscript

}

EOF

Ensure Docker and Nginx start automatically

systemctl enable docker

systemctl enable nginx

This Compose configuration assumes the repository has a root-level Dockerfile and the application listens on port 8000 inside the container

Open the VS Code terminal (Terminal → New Terminal) and run:

git clone https://github.com/betawins/cricbuzz-docker-hello-world.git

```
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz> git clone https://g
ithub.com/betawins/cricbuzz-docker-hello-world.git
Cloning into 'cricbuzz-docker-hello-world'...
remote: Enumerating objects: 127, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 127 (delta 25), reused 15 (delta 15), pack-reused 93 (from 2)
Receiving objects: 100% (127/127)
Receiving objects: 100% (127/127), 72.81 KiB | 146.00 KiB/s, done.
Resolving deltas: 100% (56/56), done.
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz>
```

Then run : - cd .\cricbuzz-docker-hello-world

- **dir**

```
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz> cd .\cricbuzz-docke
r-hello-world
>> dir


    Directory: C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuz
    z-docker-hello-world


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         11/3/2026  12:31 PM                .github
-a----         11/3/2026  12:31 PM              9 .gitattributes
-a----         11/3/2026  12:31 PM            281 .gitignore
-a----         11/3/2026  12:31 PM            183 Dockerfile
-a----         11/3/2026  12:31 PM           2116 go.mod
-a----         11/3/2026  12:31 PM          60922 go.sum
-a----         11/3/2026  12:31 PM           1508 hello_world.go
-a----         11/3/2026  12:31 PM           1140 hello_world_test.go
-a----         11/3/2026  12:31 PM          11558 LICENSE
-a----         11/3/2026  12:31 PM            151 Makefile
-a----         11/3/2026  12:31 PM           2384 README.md
```

  `Dockerfile` exists.
- **`go.mod` and `go.sum` exist.**
- **`hello_world.go` exists.**
- **`README.md` exists.**
- **This is a Go application, so we need to check how it starts and which port it listens on before configuring Docker Compose.**

<u>Check the Dockerfile</u>

Run this command inside the cricbuzz-docker-hello-world folder:

```
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuzz-docker-hell
o-world> Get-Content Dockerfile
FROM ghcr.io/vincetse/scratch
LABEL maintainer "Vince Tse <vincetse@users.noreply.github.com>"
COPY ./hello_world .
ENV GIN_MODE release
EXPOSE 8080
ENTRYPOINT ["/hello_world"]
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuzz-docker-hell
o-world>
```

````
## Usage

```
# start the container
docker run \
  --detach \
  --name hello-world \
  --publish 8000:8080 \
  ghcr.io/infrastructure-as-code/hello-world

# curl the container
curl http://0.0.0.0:8000/

# curl the health check endpoint which returns an empty response
curl http://0.0.0.0:8000/health

# curl Prometheus metrics
curl http://0.0.0.0:8000/metrics
```
````

The README confirms that the application listens inside the container on port `8080`, and it example publishes it on host port `8000`.

So our correct traffic flow will be:

Browser :8080

↓

Nginx :8080

↓

Docker host :8000

↓

Container :8080

<u>Check the Makefile</u>

Your Dockerfile contains:

```
o-world> Get-Content Makefile
all: deps test
        CGO_ENABLED=0 go build -a -o hello_world

test:
        GIN_MODE=debug go test

deps:
        go get -u
        go get github.com/stretchr/testify
```

main.tf

This creates the AWS Security Group and EC2 instance.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
```

```
}

# Security Group
resource "aws_security_group" "web_sg" {
  name        = "cricbuzz-web-sg"
  description = "Allow web traffic on port 8080"

  ingress {
    description = "Allow application traffic"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Cricbuzz-Web-SG"
  }
}

# EC2 Instance
resource "aws_instance" "cricbuzz" {
```

```
  ami                    = var.ami_id
  instance_type          = var.instance_type
  associate_public_ip_address = true

  vpc_security_group_ids = [
    aws_security_group.web_sg.id
  ]

  user_data              = file("${path.module}/user_data.sh")
  user_data_replace_on_change = true

  tags = {
    Name = "Terraform-Cricbuzz-Server"
  }
}
```

Variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "Ubuntu AMI ID"
  type        = string
  default     = "ami-0461b1436c9b3c1a3"
}
```

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

Outputs.tf

```
output "public_ip" {
  description = "Public IP of Cricbuzz EC2 instance"
  value       = aws_instance.cricbuzz.public_ip
}


output "application_url" {
  description = "Cricbuzz application URL"
  value       = "http://${aws_instance.cricbuzz.public_ip}:8080"
}
```

---

user_data.sh

This script runs automatically when EC2 launches. It installs Docker, Go, Nginx and Logrotate, builds the application, starts the Docker container, and configures the reverse proxy.

```
#!/bin/bash
set -euxo pipefail


# Update Ubuntu packages
apt-get update -y


# Install required packages
apt-get install -y docker.io nginx git golang-go logrotate


# Install Docker Compose plugin
```

```
apt-get install -y docker-compose-v2

# Start Docker
systemctl enable --now docker

# Create application directory
mkdir -p /opt/cricbuzz

# Clone Cricbuzz application
git clone https://github.com/betawins/cricbuzz-docker-hello-world.git /opt/cricbuzz/app

# Go to application directory
cd /opt/cricbuzz/app

# Build Go application
CGO_ENABLED=0 go build -a -o hello_world

# Create Docker Compose file
cat > docker-compose.yml <<'EOF'
services:
  cricbuzz:
    build: .
    container_name: cricbuzz-app
    ports:
      - "127.0.0.1:8000:8080"
    restart: unless-stopped
EOF

# Build and start Docker container
```

```
docker compose up -d --build

# Configure Nginx reverse proxy
cat > /etc/nginx/sites-available/cricbuzz <<'EOF'
server {
   listen 8080;
   server_name _;

   access_log /var/log/cb.log combined;
   error_log /var/log/cb-error.log warn;

   location / {
      proxy_pass http://127.0.0.1:8000;

      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
   }
}
EOF

# Enable Nginx configuration
rm -f /etc/nginx/sites-enabled/default

ln -s /etc/nginx/sites-available/cricbuzz \
/etc/nginx/sites-enabled/cricbuzz

# Test Nginx configuration
nginx -t
```

Start Nginx

systemctl enable --now nginx

systemctl restart nginx

Configure Logrotate

cat > /etc/logrotate.d/cricbuzz <<'EOF'

/var/log/cb.log {

daily

rotate 7

compress

missingok

notifempty

create 0640 www-data adm

sharedscripts

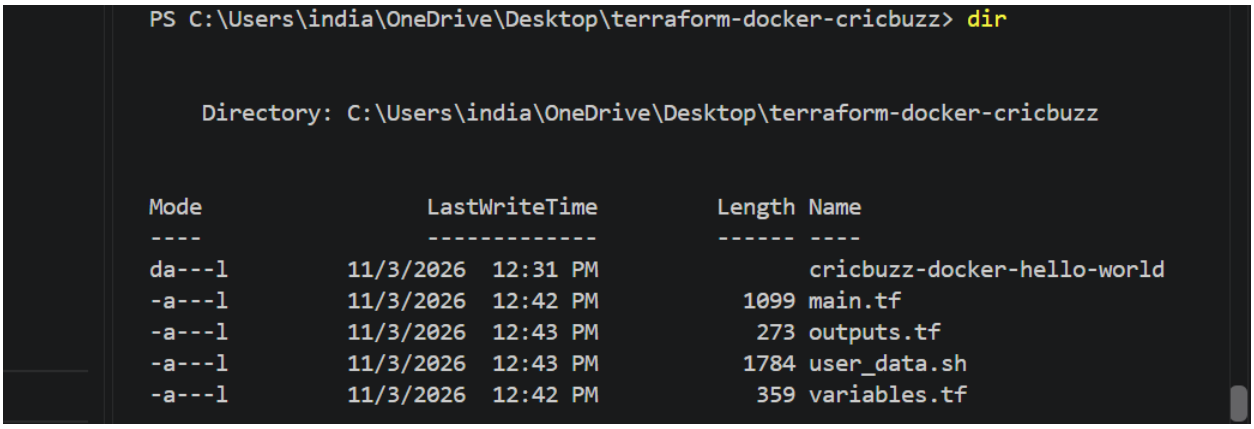
postrotate

nginx -s reopen

endscript

}

EOF



```
o-world> terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuzz-docker-hell
o-world> terraform fmt
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuzz-docker-hell
o-world>
```

```
o-world> terraform validate
Success! The configuration is valid.

PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz\cricbuzz-docker-hell
o-world>
```

Create the infrastructure: terraform apply

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.cricbuzz: Creating...
aws_instance.cricbuzz: Still creating... [00m10s elapsed]
aws_instance.cricbuzz: Creation complete after 16s [id=i-026b3088985b8a72f]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

application_url = "http://44.200.24.164:8080"
public_ip = "44.200.24.164"
PS C:\Users\india\OneDrive\Desktop\terraform-docker-cricbuzz>
```